Capítulo 1

Desarrollo e implementación

El desarrollo de FormQuid convirtió el análisis inicial de la plataforma en una aplicación que permite construir encuestas, reutilizar contenido, trabajar de forma colaborativa y consultar las respuestas obtenidas. La dificultad principal no residía únicamente en disponer de formularios para introducir datos. Era necesario mantener la coherencia entre el catálogo y las encuestas que lo utilizan, distinguir los permisos de cada participante y conservar la relación entre las preguntas publicadas y sus respuestas.

El capítulo organiza la explicación según la planificación de las iteraciones. En cada una se describe el objetivo funcional, las páginas y componentes implicados y la forma en que las operaciones llegan al servidor y a la base de datos. Esta organización permite seguir la evolución desde la preparación de junio y julio hasta las funcionalidades de participación y análisis, y comprender cómo los módulos terminan formando una misma aplicación.

1.1. Diseño y preparación técnica

1.1.1. Objetivo de la fase

El diseño inicial y la preparación técnica se abordaron de forma progresiva durante junio y julio, en paralelo a la investigación y el análisis. Se definieron el alcance funcional, la arquitectura general y las tecnologías adecuadas para la plataforma. La configuración concreta del proyecto se concentró en la semana del 13 al 19 de julio, antes del Sprint 0.

Durante esta fase también se profundizó en el funcionamiento de Spring Boot, que no se conocía en profundidad al inicio del proyecto. Este aprendizaje fue necesario para comprender cómo se relacionan los controladores HTTP, los servicios de aplicación, los repositorios y las entidades persistentes. Además, se configuraron los repositorios y se estableció el entorno de trabajo que serviría de base para las siguientes iteraciones.

1.1.2. Arquitectura y resultados de la fase

La aplicación se organizó en un cliente React, un servidor Spring Boot y una base de datos PostgreSQL. El cliente se ocupa de la navegación, la composición visual de las encuestas y la interacción inmediata con el usuario. El servidor mantiene las reglas de negocio, comprueba los permisos y transforma las peticiones en operaciones persistentes. PostgreSQL conserva tanto la estructura de las encuestas como las participaciones y los datos necesarios para su administración.

La comunicación entre cliente y servidor se articula mediante una API bajo el prefijo `/api/v1`.

Se utilizan objetos de transferencia de datos para expresar las entradas y las respuestas de cada operación. Así, la representación enviada a una página no depende de exponer directamente una entidad JPA ni de trasladar al navegador todas sus relaciones. Esta separación resulta especialmente útil en las encuestas: una vista de listado necesita un resumen, mientras que el editor requiere contenido ordenado, configuración y permisos sobre el recurso.

El servidor se distribuye por áreas funcionales, entre ellas `auth`, `user`, `question`, `block`, `template`, `content`, `survey`, `collaboration`, `participation` y `statistics`. Dentro de cada área se separan los puntos de entrada HTTP, la lógica de aplicación y el acceso a datos. Esta organización permite localizar una funcionalidad siguiendo su recorrido completo y evita concentrar las reglas del sistema en los controladores.

En el cliente se diferencian las páginas asociadas a rutas, los componentes reutilizables, los módulos de acceso a la API y las utilidades que adaptan los datos. Por ejemplo, una página de edición coordina la carga y el guardado, pero delega la construcción visual en componentes especializados. A su vez, estos componentes trabajan con un estado editable que se transforma en el formato esperado por el servidor al guardar.

Como resultado de la fase se dispuso de una arquitectura de referencia y de un entorno preparado para evolucionar por incrementos. Las decisiones sobre el modelo de datos continuaron revisándose al implementar los primeros recorridos reales, especialmente cuando aparecieron necesidades de reutilización, colaboración y conservación de respuestas.

1.1.3. Planificación de la fase y de las iteraciones

La Tabla 1.1 recoge la planificación utilizada para estructurar el desarrollo. El trabajo previo de junio y julio permitió llegar al Sprint 0 con las decisiones generales planteadas, mientras que las iteraciones posteriores concretaron progresivamente el comportamiento de la plataforma.

Tabla 1.1: Organización temporal y funcional del desarrollo.

Iteración	Fechas	Objetivo principal
Preparación	Junio y julio; configuración del 13 al 19 de julio	Análisis, diseño general, aprendizaje de Spring Boot y preparación del entorno.
Sprint 0	20 de julio–2 de agosto	Estructura del servidor, conexión con la base de datos y seguridad básica.
Sprint 1	3–23 de agosto	Autenticación, usuarios y catálogo inicial; revisión del modelo y del backlog.
Sprint 2	24 de agosto–6 de septiembre	Catálogo reutilizable, constructores y creación inicial de encuestas.
Sprint 3	7–20 de septiembre	Edición, configuración, colaboración y supervisión de encuestas.
Sprint 4	21 de septiembre–4 de octubre	Participación, importación, galería y consulta de resultados.

1.2. Sprint 0: Base del backend

1.2.1. Objetivo de la iteración

Del 20 de julio al 2 de agosto se establecieron la estructura del servidor, la conexión inicial con la base de datos y los mecanismos básicos de seguridad. Esta iteración proporcionó el soporte

común necesario para implementar posteriormente las historias de autenticación, catálogo y gestión de encuestas.

1.2.2. Estructura del servidor y persistencia

La base técnica se construyó con Spring Boot y Java, utilizando JPA para representar y persistir las entidades del dominio. Los repositorios encapsulan las consultas y los servicios coordinan las operaciones que afectan a varias entidades. Por ejemplo, la creación de una estructura de encuesta exige guardar un contenedor y sus elementos ordenados; esa responsabilidad corresponde al servicio, mientras que el controlador recibe y valida la forma de la petición.

La configuración de conexión y de ejecución se separa de la lógica funcional. Las propiedades del servidor permiten ajustar la conexión, los orígenes admitidos y los directorios de almacenamiento sin modificar los servicios. Los valores sensibles, como el secreto utilizado para los JWT y las credenciales de servicios externos, se proporcionan mediante configuración del entorno.

La evolución del esquema se recoge en migraciones de Flyway. Este mecanismo permite que los cambios posteriores en contenido, colaboraciones o participaciones se expresen como modificaciones identificables de la base de datos. Las restricciones de persistencia complementan las comprobaciones de los servicios: las relaciones mantienen la vinculación entre entidades y los índices y restricciones específicas respaldan reglas como la ausencia de participaciones duplicadas dentro del alcance definido por la aplicación.

1.2.3. Seguridad y contrato de errores

La configuración de seguridad distingue las operaciones públicas de las que requieren una sesión autenticada y de aquellas reservadas a determinados roles. Sobre esta base se incorpora la autenticación mediante JWT. El filtro `JwtAuthenticationFilter` obtiene la credencial de la petición y establece el contexto de seguridad que permite a los servicios identificar al usuario que realiza la operación.

Esta identificación no sustituye las comprobaciones sobre recursos concretos. Una persona puede tener el rol de editor y carecer de permiso para modificar una encuesta determinada. Por ello, la seguridad combina restricciones generales de acceso con reglas de propiedad y colaboración en los servicios. Esta distinción se convierte en una parte central del editor y de la consulta de resultados en las siguientes iteraciones.

También se establece una respuesta homogénea ante errores mediante `ApiException` y `GlobalExceptionHandler`. Las operaciones pueden comunicar un problema de validación, una falta de permisos, un recurso inexistente o un conflicto con el estado actual. El cliente recibe un estado HTTP y un contenido interpretable, en lugar de depender de mensajes aislados o de excepciones internas del servidor.

1.2.4. Resultado de la iteración

El resultado fue una base sobre la que podían añadirse funcionalidades sin repetir la configuración de seguridad, persistencia y tratamiento de errores. La distribución de responsabilidades permitió abordar el siguiente sprint a partir de recorridos completos de usuario, manteniendo las reglas del dominio en el servidor.

1.3. Sprint 1: Acceso y catálogo inicial

1.3.1. Objetivo e historias de usuario relacionadas

Entre el 3 y el 23 de agosto se desarrolló el acceso a la plataforma, la gestión de cuentas y el primer nivel del catálogo. En la numeración consolidada del backlog, este bloque se relaciona con HU.4 a HU.8, dedicadas a autenticación, registro, cuenta, panel y administración de usuarios, y con el inicio de HU.9, relativa a preguntas fijas. La duración de tres semanas responde a la revisión del modelo y de los requisitos descrita al final de la iteración.

1.3.2. Inicio de sesión y recuperación del estado de autenticación

El recorrido de acceso comienza en la página de inicio de sesión. El cliente envía las credenciales a través de `authApi`, y `AuthService` normaliza el correo y delega su comprobación en el mecanismo de autenticación de Spring Security. Las contraseñas se almacenan mediante un hash BCrypt. Cuando las credenciales son correctas, el servidor genera el JWT y lo entrega mediante una cookie HttpOnly.

El uso de esta cookie permite que el navegador acompañe las peticiones posteriores sin que los componentes React tengan que leer o almacenar el token. El estado compartido del cliente se mantiene en `AuthProvider`, definido en `AuthContext.jsx`. Este proveedor conserva los datos de usuario necesarios para la interfaz y distingue entre un estado todavía desconocido, una sesión autenticada, una sesión anónima y un error de recuperación.

Al recargar una página, el contexto consulta `/users/me` para recuperar la sesión. Mientras esta consulta está pendiente, las rutas protegidas esperan su resolución; así se evita interpretar una carga inicial como una ausencia de permisos. Una respuesta de no autenticado limpia el estado de sesión, mientras que un fallo de comunicación se trata como un error distinto. El comportamiento de navegación depende, por tanto, de una comprobación efectiva del servidor.

El archivo `router.jsx` reúne la navegación y utiliza `RequireAuth` y `RequireRole` para proteger los recorridos del cliente. Tras el acceso, `PanelPage` presenta las funciones disponibles según el perfil. Los editores acceden a sus encuestas y colaboraciones; los perfiles administrativos disponen además de las secciones de revisión y supervisión correspondientes.

El módulo `apiClient.js` centraliza las llamadas HTTP. Incluye las credenciales en las peticiones, interpreta las respuestas JSON y transforma los errores en objetos `ApiError`. Para las operaciones que modifican datos obtiene el token CSRF y lo envía en la cabecera prevista por el servidor. También agrupa peticiones GET idénticas que están simultáneamente en curso, compartiendo su promesa; esta medida evita solicitudes concurrentes redundantes, sin convertir la respuesta en una caché permanente.

1.3.3. Solicitud de cuenta y revisión administrativa

El registro se plantea como un procedimiento de solicitud y revisión. La página de solicitud recoge los datos personales, la institución, el país y la motivación. `RegistrationRequestService` comprueba que el correo no corresponda ya a una cuenta ni a una solicitud pendiente y almacena la contraseña mediante hash desde este primer paso.

La solicitud llega a `RegistrationRequestsPage`, donde el administrador puede consultar su contenido y decidir sobre ella. La tarjeta de solicitud y los diálogos de confirmación separan la lectura de los datos de la acción definitiva. Al aprobar, el servicio crea el usuario con rol `EDITOR`, reutiliza el hash almacenado y elimina la solicitud. Estas operaciones se coordinan de forma transaccional para no dejar una aprobación parcialmente aplicada.

Al rechazar se comunica la decisión y se elimina la solicitud pendiente. En este diseño, la existencia del registro representa una solicitud todavía por resolver; no se mantiene una tabla adicional de historial de estados. El proceso se conecta con el módulo de notificaciones y con el envío de correo, cuya implementación transversal se detalla en el Sprint 4.

1.3.4. Gestión de cuenta, contraseñas y roles

`AccountPage` concentra la consulta y modificación de la cuenta. Desde ella se actualizan los datos de perfil y se gestionan cambios sensibles, como el correo o la contraseña. `UserService` solicita la contraseña actual para confirmar estas operaciones y realiza las comprobaciones necesarias antes de persistirlas. La fotografía se gestiona mediante un servicio de almacenamiento específico, evitando incorporar el fichero binario al objeto de usuario enviado habitualmente al cliente.

La recuperación de contraseña utiliza un recorrido independiente, con las páginas `PasswordRecoveryPage` y `ResetPasswordPage`. `PasswordResetService` genera un token aleatorio, guarda su hash y fija una caducidad de treinta minutos. La solicitud mantiene una respuesta genérica cuando el correo no corresponde a una cuenta y limita la repetición inmediata de envíos. Al confirmar el enlace, el servidor comprueba el token y su vigencia, actualiza el hash de la contraseña y elimina el token utilizado.

La administración de roles se realiza desde `UsersManagementPage`, reservada al superadministrador. No se limita a actualizar un campo: el servidor protege la existencia de al menos un usuario `SUPERADMIN`. Para ello combina comprobaciones y bloqueos durante las operaciones que podrían retirar el último perfil de este tipo.

La eliminación de una cuenta también depende de las relaciones del dominio. Si el usuario conserva encuestas en propiedad, el servidor impide la baja. Cerrar una encuesta no elimina esa propiedad; es necesario resolverla mediante su eliminación o transferencia, según corresponda. De esta forma, la gestión de usuarios no deja encuestas sin una persona responsable.

1.3.5. Primer nivel del catálogo: preguntas fijas

Las preguntas fijas constituyen la unidad reutilizable más pequeña del catálogo. Su representación incluye el enunciado, una anotación opcional, el tipo de respuesta y una configuración adaptada a ese tipo. `QuestionBuilder` ofrece una interfaz común para construirlas, mientras que `QuestionService` comprueba las reglas y mantiene sus opciones, filas de matriz y ajustes.

Se contemplan preguntas de texto, numéricas, de elección única, de elección múltiple, de escala, de matriz de escala y booleanas. Cada tipo exige datos diferentes: una pregunta de texto puede limitar la longitud; una numérica puede establecer mínimo, máximo e incremento; una selección múltiple puede restringir el número de opciones elegidas; y una matriz define las filas sobre las que se responderá utilizando una escala.

La configuración se expresa mediante estructuras específicas, como `QuestionSettingsDto`, en lugar de añadir campos arbitrarios a la interfaz. El servicio comprueba la coherencia de rangos, opciones y posiciones. Esto permite que las mismas reglas sean utilizadas cuando una pregunta se crea desde el catálogo, dentro de un bloque o como parte de una encuesta.

El contenido fijo del catálogo se conserva con evaluación neutral. La dirección y el peso de una pregunta pertenecen a su uso evaluativo dentro de una encuesta, explicado posteriormente. Esta decisión evita que una pregunta reutilizable imponga de forma automática una interpretación estadística a todos los contextos en los que se incorpore.

1.3.6. Revisión del modelo e incidencias resueltas

Antes de comenzar la implementación funcional del Sprint 1 se revisó la primera versión del modelo de datos definida en el Sprint 0. Se detectaron entidades poco ajustadas al funcionamiento real de la plataforma, atributos con alta probabilidad de permanecer vacíos y relaciones cuya utilidad no justificaba su mantenimiento.

La revisión simplificó las etiquetas, que pasaron a almacenarse como información propia de cada encuesta, sin una entidad independiente. También simplificó las notificaciones mediante una tabla vinculada al receptor y amplió las solicitudes de catálogo para admitir preguntas fijas, bloques y plantillas. Finalmente, sustituyó la mezcla de estados de encuesta por tres dimensiones independientes: ciclo de vida, modo de acceso y visibilidad.

Estos cambios afectaron al Product Backlog. El borrador de revisión partía de veintidós historias y contemplaba una reorganización en veinticuatro; la relación consolidada consultada contiene veintitrés historias. La Tabla 1.2 conserva la numeración original para explicar qué cambió, por lo que sus identificadores no deben confundirse con los utilizados para relacionar las funcionalidades de los sprints con el backlog consolidado.

Tabla 1.2: Cambios sobre las historias originales tras la revisión del modelo y los requisitos.

Historia original	Cambio realizado
HU.1. Visualización de encuestas públicas	Se reformula como consulta de encuestas disponibles, atendiendo a su apertura, visibilidad y condiciones de acceso.
HU.2. Acceso a encuestas	Se sustituye la mezcla de borrador, público, privado y oculto por ciclo de vida, acceso y visibilidad independientes.
HU.3. Visualizar y responder	Se amplía con validación de respuestas, creación de participaciones y relación con las estadísticas.
HU.4. Inicio de sesión	Se mantiene e incorpora la autenticación mediante JWT y la navegación correspondiente al rol.
HU.5. Panel del administrador	Se transforma en un panel según el perfil y se separa de la gestión de usuarios y roles.
HU.6. Preguntas fijas	Se mantiene la gestión de preguntas y se elimina su asociación con etiquetas.
HU.7. Solicitud de registro	Se distinguen la presentación de la solicitud y su revisión administrativa; el backlog consolidado las reúne en el proceso de registro.
HU.8. Creación de encuestas	Se amplía con metadatos, imagen, etiquetas y creación desde cero o desde plantilla.
HU.9. Edición y reestructuración	Se precisa que la estructura puede modificarse mientras no existan participaciones registradas, además de cumplirse los permisos de edición.
HU.10. Personalización	Se integra en el editor y en la configuración de la encuesta.
HU.11. Previsualización	Se conserva como funcionalidad diferenciada, sin registrar participaciones.
HU.12. Gestión de estados	Se redefine mediante ciclo de vida, modo de acceso y visibilidad.
HU.13 y HU.14. Coautoría	Se reorganizan en colaboración, invitaciones, solicitudes de acceso y publicación colaborativa.
HU.15. Estadísticas	Se amplía con métricas agregadas, análisis por pregunta y consulta de participaciones.

Historia original	Cambio realizado
HU.16. Importación desde Word	Se conserva el objetivo de incorporar contenido desde un documento estructurado y de comunicar sus errores. La implementación final utiliza una plantilla ODT.
HU.17. Permisos administrativos	Se integra en la gestión de usuarios y roles.
HU.18. Notificaciones	Se amplía con eventos de la aplicación, estado de lectura, correo y eliminación manual.
HU.19. Baja de usuario	Se amplía a la gestión completa de la cuenta, incluyendo las restricciones derivadas de la propiedad de encuestas.
HU.20. Plantillas y bloques	Se separa en gestión de bloques, gestión de plantillas y constructores de catálogo.
HU.21. Nuevos bloques	Se sustituye por solicitudes de elementos de catálogo de los tres tipos admitidos.
HU.22. Etiquetado	Deja de ser una historia independiente; las etiquetas pertenecen únicamente a las encuestas.

1.3.7. Planificación de la iteración y resultado

La revisión del modelo y del backlog prolongó el Sprint 1 una semana respecto a las dos previstas. La comparación puede expresarse directamente: la duración inicial era de dos semanas y la duración de la iteración se amplió a tres, comprendidas entre el 3 y el 23 de agosto. Esta ampliación permitió ajustar el modelo antes de extenderlo a la composición de bloques, plantillas y encuestas.

El resultado de la iteración fue un recorrido de acceso y gestión de usuarios conectado con un catálogo inicial. La revisión tuvo consecuencias concretas sobre el código posterior: las etiquetas dejaron de requerir un módulo independiente, las solicitudes de catálogo pudieron compartir un mismo procedimiento y la configuración de una encuesta dejó de depender de un único estado ambiguo.

1.4. Sprint 2: Catálogo y creación de encuestas

1.4.1. Objetivo e historias de usuario relacionadas

Del 24 de agosto al 6 de septiembre se completó la composición del catálogo y su incorporación a las encuestas. El trabajo se relaciona con HU.9 a HU.15: preguntas, bloques, plantillas, constructores, propuestas de catálogo, consulta de detalles y creación de encuestas. El objetivo era que el contenido pudiera definirse una vez y reutilizarse sin perder la posibilidad de adaptarlo a cada encuesta.

1.4.2. Listado, búsqueda y detalle de elementos

`CatalogPage` proporciona una misma estructura de página para preguntas, bloques y plantillas. Una configuración por sección determina qué función de `catalogApi` carga los datos, qué propiedades se presentan y qué operaciones corresponden a cada tipo. Así se comparten la búsqueda, el filtrado por categoría, la ordenación y la paginación, manteniendo las diferencias funcionales de los elementos.

La página distingue los conjuntos que muestra y conserva su paginación. La búsqueda

incorpora un pequeño retardo para evitar una consulta por cada pulsación, y utiliza una versión de petición para impedir que una respuesta antigua sustituya resultados más recientes. Estas medidas son relevantes cuando se modifican rápidamente el texto o los filtros.

`ElementDetailPage` permite examinar un elemento antes de utilizarlo. En un bloque o una plantilla, el detalle incluye su estructura, de modo que la elección no depende únicamente del título. Las páginas de creación y edición reutilizan los mismos constructores, por lo que la representación del contenido se mantiene coherente entre la consulta y la modificación.

Las categorías se gestionan como datos del catálogo mediante su servicio y la página `CategoriesManagementPage`. La composición comprueba la compatibilidad entre la categoría del contenedor y la de los elementos seleccionados. Además, se contempla el tratamiento específico de la categoría de datos personales, que no se traslada automáticamente como categoría de una encuesta.

Los permisos se aplican también en el servidor. Por ejemplo, `QuestionService` permite gestionar una pregunta a su creador o al superadministrador. Mostrar u ocultar una acción en el catálogo ayuda a orientar la interfaz, pero la autorización efectiva se realiza al atender la petición.

1.4.3. Modelo de contenido y construcción por composición

La estructura de los documentos se representa mediante `ContentItem`. Un elemento de contenido recoge el tipo, la posición, la obligatoriedad cuando corresponde y su vinculación con el contenedor. Este puede ser una encuesta, una plantilla o un bloque. El texto informativo se guarda como contenido propio, mientras que las preguntas y los bloques se relacionan con sus entidades específicas.

Un bloque puede contener textos y preguntas, y debe incorporar al menos una pregunta. Una plantilla admite textos, preguntas y bloques. No se implementa una anidación ilimitada de bloques, lo que mantiene una estructura comprensible tanto al editar como al responder. El orden se almacena explícitamente y no depende del identificador asignado por la base de datos.

En el cliente, `QuestionBuilder` construye la unidad básica; `BlockBuilder` organiza preguntas y textos; y `TemplateBuilder` combina estos elementos con bloques. Los constructores abren formularios o diálogos según el elemento que se esté añadiendo o modificando, y notifican los cambios al componente que mantiene el estado de la composición.

`CatalogSidebar` acerca los elementos reutilizables al área de edición. Desde este panel pueden seleccionarse preguntas y bloques, consultar sus detalles e incorporarlos a la estructura. La interacción incluye arrastrar y soltar, junto con controles de ordenación. El constructor distingue las incorporaciones procedentes del catálogo de los movimientos de elementos que ya están dentro del documento.

El movimiento entre el nivel principal y un bloque requiere actualizar tanto el contenedor como la posición. Las utilidades de los constructores normalizan esta estructura antes de serializarla. También distinguen identificadores locales, necesarios mientras un elemento todavía no se ha guardado, de los identificadores persistentes recibidos de la API.

La reutilización más relevante aparece en `SurveyBuilder`: este componente configura `TemplateBuilder` para el contexto de una encuesta, oculta sus datos generales y activa la configuración evaluativa. El siguiente fragmento reducido conserva esa relación del código:

```
<TemplateBuilder
  data={data}
  onChange={onChange}
  readOnly={readOnly}
  showGeneralInfo={false}
  containerLabel="encuesta"
  includeEvaluation
/>
```

Por tanto, el editor de encuestas comparte el mecanismo de composición con las plantillas. Las diferencias se expresan mediante propiedades y mediante la coordinación de la página, que añade metadatos, configuración, colaboración y operaciones sobre la encuesta.

1.4.4. Copias del catálogo e independencia de las encuestas

Reutilizar un elemento no equivale a mantener una referencia mutable a su contenido. Si una encuesta dependiera directamente de una pregunta de catálogo, una modificación administrativa podría cambiar el significado de las respuestas ya recogidas. Para evitarlo, los servicios construyen copias del contenido al incorporarlo a los documentos que lo utilizan.

`ContentService` coordina la incorporación de preguntas y bloques. La copia de un bloque recorre sus textos y preguntas, conserva el orden y genera las entidades que corresponden al nuevo contexto. La copia de una pregunta incluye sus ajustes, opciones y filas de matriz. Así, modificar después la copia de una encuesta no altera el elemento original del catálogo.

La misma idea se aplica al crear una encuesta desde plantilla: `TemplateService` proporciona su estructura y el nuevo documento recibe su propio contenido. El catálogo actúa como punto de partida reutilizable, mientras que la encuesta conserva autonomía para adaptar el enunciado, la obligatoriedad o la evaluación.

El servidor valida que cada elemento de entrada exprese un origen coherente. Una pregunta puede proceder de una referencia de catálogo o de una definición local, pero no de ambas simultáneamente. También comprueba las posiciones y los textos obligatorios. De este modo, el guardado no confía únicamente en que el constructor haya producido una estructura correcta.

1.4.5. Propuestas de incorporación al catálogo

`SuggestElementPage` permite proponer preguntas fijas, bloques y plantillas. La selección del tipo determina el constructor que aparece y la transformación de sus datos. La propuesta incluye categoría y justificación, de modo que la revisión administrativa dispone del contenido y del motivo de su incorporación.

`ElementRequestService` interpreta el contenido de la solicitud según su tipo y lo convierte en el DTO correspondiente. A continuación reutiliza las validaciones de los servicios de preguntas, bloques o plantillas. La solicitud pendiente conserva esa propuesta, que todavía no se incorpora al catálogo disponible.

Desde `ElementRequestsPage`, el administrador consulta el detalle y aprueba o rechaza. La aprobación invoca los mismos servicios que crean los elementos ordinarios del catálogo y elimina la solicitud resuelta dentro de la operación transaccional. Esta reutilización evita que el contenido aprobado siga un procedimiento de persistencia diferente del contenido creado directamente. La decisión se comunica al solicitante mediante el módulo de notificaciones.

1.4.6. Creación inicial de una encuesta

`CreateSurveyPage` coordina el inicio de una encuesta. `SurveyOriginModal` permite seleccionar el origen: una estructura vacía, una plantilla o el recorrido de importación que se completa en el Sprint 4. La página recoge el título, la descripción, la categoría, las etiquetas y la configuración inicial correspondiente.

Las etiquetas se normalizan en el cliente, eliminando repeticiones y aplicando los límites previstos: hasta diez etiquetas y cincuenta caracteres por etiqueta. Su finalidad es describir la encuesta; no se crea un catálogo independiente de etiquetas. La fuente tipográfica se configura para el conjunto del documento, manteniendo una presentación común entre sus preguntas.

El guardado comienza con la creación del borrador mediante `SurveyService`. Si se selecciona una plantilla, se incorpora su estructura mediante el procedimiento de copia. Después, cuando hay una portada elegida, el cliente realiza su carga y navega hacia la página de edición.

La creación y la subida de portada son peticiones diferentes. `CreateSurveyPage` conserva una referencia a la encuesta ya creada para que un fallo en la segunda operación no provoque otra encuesta al reintentar. También utiliza un bloqueo local durante el envío para evitar solicitudes repetidas por pulsaciones consecutivas. Este tratamiento refleja un caso concreto en el que la coordinación de la interfaz debe considerar que una parte del proceso ya puede haberse completado en el servidor.

1.4.7. Resultado de la iteración

Al finalizar este bloque funcional, FormQuid disponía de un catálogo compuesto y de un recorrido de creación conectado con él. Los constructores y las utilidades de transformación resolvieron la edición de estructuras heterogéneas, mientras que las copias persistentes separaron el contenido reutilizable del contenido de cada encuesta.

1.5. Sprint 3: Edición y gestión de encuestas

1.5.1. Objetivo e historias de usuario relacionadas

Del 7 al 20 de septiembre se amplió la gestión de las encuestas mediante edición, previsualización, configuración, colaboración y supervisión. Estas funciones se corresponden con HU.16 a HU.20. El trabajo se centró en completar el recorrido posterior a la creación del borrador y en definir qué operaciones puede realizar cada persona sobre una misma encuesta.

1.5.2. Coordinación del editor y guardado de la estructura

`EditSurveyPage` actúa como punto de coordinación. Organiza el trabajo en las pestañas de encuesta, colaboraciones y configuración, carga la información del recurso y calcula qué acciones debe ofrecer según los permisos recibidos y la relación del usuario con él. La estructura se delega en `SurveyBuilder`; la página conserva la responsabilidad de guardar y de mantener sincronizados los datos generales.

Las utilidades de `surveyBuilderUtils` transforman el contenido recibido en un estado editable y generan la representación que espera la API. Este paso permite manejar de forma uniforme elementos existentes y elementos recién añadidos, sin exigir que cada componente visual conozca el modelo completo de persistencia.

Al guardar la estructura, el cliente llama a la operación de sustitución de contenido y actualiza después los metadatos que corresponden. Finalmente recupera la encuesta para sincronizar el

estado local con la versión persistida. Son solicitudes coordinadas por la página, por lo que no constituyen una única transacción que abarque todas las acciones del navegador.

En el servidor, `ContentService.replaceSurveyContent` valida la nueva composición y realiza la sustitución dentro de una transacción. El procedimiento elimina el contenido anterior y reconstruye los elementos ordenados a partir de la petición. Esta estrategia resulta apropiada para guardar una composición completa, pero hace especialmente importante la regla que protege las encuestas con respuestas.

Antes de modificar la estructura, el controlador comprueba los permisos mediante `SurveyService.canEditSurvey` y aplica `ensureStructureEditable`. Este último método consulta si existen participaciones y rechaza el cambio cuando ya se han registrado. La restricción protege la correspondencia entre las preguntas, sus opciones y las respuestas existentes. El criterio no es simplemente que la encuesta se haya abierto alguna vez: la presencia de participaciones es la condición específica que impide reestructurarla.

La página detecta cambios pendientes en estructura, etiquetas y configuración. Para evitar pérdidas involuntarias utiliza el bloqueo de navegación de React Router y el aviso de salida del navegador. Las acciones que cambian el ciclo de vida también consideran los cambios sin guardar, de modo que el usuario no publique una versión distinta de la que está viendo sin resolver antes esa situación.

1.5.3. Ciclo de vida, acceso y visibilidad

La configuración separa tres decisiones que inicialmente estaban mezcladas. El ciclo de vida establece si la encuesta está en borrador, abierta o cerrada. El modo de acceso determina si puede responderse públicamente, mediante contraseña o con autenticación. La visibilidad controla si aparece listada o si se accede mediante su enlace.

Tabla 1.3: Dimensiones de configuración de una encuesta.

Dimensión	Valores del modelo	Efecto funcional
Ciclo de vida	`DRAFT`, `OPEN`, `CLOSED`	Controla la preparación, la apertura y el cierre de la recogida de respuestas.
Acceso	`PUBLIC`, `PASSWORD_PROTECTED`, `AUTHENTICATED`	Establece la condición que debe cumplir quien va a responder.
Visibilidad	`LISTED`, `UNLISTED`	Determina la inclusión en el listado disponible de encuestas.

Una encuesta abierta y no listada puede distribuirse mediante un enlace. Del mismo modo, figurar en un listado no elimina una contraseña o un requisito de autenticación. Esta separación permite que `SurveyService` evalúe cada condición de forma explícita y que la interfaz presente controles comprensibles.

La apertura comprueba que exista una estructura válida con preguntas y establece el código público utilizado por el enlace. Las operaciones de ciclo de vida y política de acceso se reservan a la persona responsable según las reglas del servicio; los permisos de edición de un colaborador no se interpretan como una cesión automática de todas las decisiones de publicación.

1.5.4. Personalización y previsualización

La configuración visual incluye la portada y la familia tipográfica de la encuesta. Entre las opciones previstas se encuentran fuentes orientadas a distintas preferencias de lectura, como

OpenDyslexic y Lexend, junto con las alternativas generales. La elección se aplica al documento para mantener consistencia entre textos, preguntas y bloques.

`SurveyPreviewModal` utiliza el componente `Survey` en modo de previsualización. De esta manera, el autor revisa el orden, la numeración, las opciones y los controles con el mismo componente que se emplea para responder. La diferencia se establece en el modo de ejecución: la previsualización no envía una participación ni incrementa los resultados.

Esta reutilización también permite comprobar visualmente cómo afectan los cambios de estructura y configuración al formulario completo. Los textos informativos se presentan dentro del recorrido, los bloques agrupan sus preguntas y la numeración de preguntas se mantiene a través de los distintos contenedores.

1.5.5. Colaboradores, invitaciones y solicitudes

La colaboración se gestiona desde `CollaborationsPage` y desde `CollaborationsPanel`, integrado en el editor. El modelo diferencia la relación de colaboración ya aceptada de las solicitudes pendientes. Por tanto, recibir una invitación no concede por sí solo acceso definitivo a la encuesta.

`CollaborationService` coordina invitaciones, solicitudes de acceso y decisiones sobre ellas. Los permisos de colaboración distinguen consulta y edición mediante `VIEW` y `EDIT`. El servicio verifica quién puede iniciar o resolver cada operación, y la relación aceptada se utiliza después para determinar las capacidades sobre el recurso.

Por ejemplo, cuando una persona con permiso de edición abre la encuesta, la página puede ofrecerle el constructor. Al guardar, el servidor vuelve a comprobar su relación con esa encuesta y la ausencia de participaciones si se modifica la estructura. El rol global de editor habilita el área de trabajo, mientras que la colaboración concreta determina qué puede hacer en ese documento.

La publicación colaborativa incorpora una solicitud específica. Un colaborador puede promover la apertura, pero la decisión sigue el procedimiento de aprobación previsto. Al aceptarla se reutiliza la lógica de apertura de `SurveyService`, incluidas sus comprobaciones sobre el contenido. Así se evita que publicar desde una solicitud tenga condiciones diferentes de publicar desde la configuración ordinaria.

1.5.6. Transferencia de propiedad y supervisión

La transferencia de propiedad relaciona los módulos de cuenta, encuestas y colaboración. `UserService.transferSurveyOwnership` exige que la operación la realice el propietario, confirma su contraseña y comprueba que el destinatario sea un colaborador existente. Después cambia la propiedad, retira la relación de colaboración del nuevo propietario y convierte al anterior en colaborador con permiso de edición.

Esta operación mantiene la continuidad del trabajo y permite resolver la situación de una cuenta que conserva encuestas antes de solicitar su baja. Las notificaciones informan a las personas implicadas, pero el cambio de propiedad se resuelve en el servidor y no depende de que se haya leído el aviso.

`AdminSurveysPage` proporciona el recorrido de supervisión administrativa. Permite localizar encuestas y consultar su situación, y desde él se accede a la vista correspondiente del editor o de los resultados. La interfaz distingue esta supervisión de la edición ordinaria: disponer de un rol administrativo no se convierte en una autorización general para reescribir cualquier contenido.

Las intervenciones administrativas se atienden mediante operaciones específicas. En particular, ocultar una encuesta del listado es una acción distinta de cambiar su estructura o

apropiarse de ella. Mantener estas operaciones separadas hace que los permisos respondan a responsabilidades concretas del sistema.

1.5.7. Resultado de la iteración

La encuesta dejó de ser únicamente un documento creado por una persona y pasó a disponer de un ciclo de gestión completo. El editor, la previsualización y las reglas de colaboración comparten la misma estructura persistente, mientras que las comprobaciones de propiedad, permisos y participaciones protegen las operaciones que pueden modificarla.

1.6. Sprint 4: Participación, importación, galería y estadísticas

1.6.1. Objetivo e historias de usuario relacionadas

Del 21 de septiembre al 4 de octubre se completó el recorrido desde la publicación hasta la consulta de resultados. Este bloque se relaciona con HU.1 a HU.3, relativas a disponibilidad, acceso y respuesta; HU.21, sobre notificaciones; HU.22, sobre resultados; y HU.23, sobre importación. La galería amplía la personalización de las encuestas creada en las iteraciones anteriores.

1.6.2. Localización y acceso a una encuesta publicada

`MainPage` presenta las encuestas abiertas y listadas que corresponden al recorrido de consulta. Además, cada encuesta publicada puede alcanzarse mediante su código público, compuesto por doce caracteres alfanuméricos. La ruta de `PublicSurveyPage` interpreta ese código y coordina la carga del formulario.

La página espera a conocer el estado de autenticación antes de seleccionar la operación de consulta adecuada. Cuando el acceso requiere contraseña, muestra el paso de desbloqueo y solicita al servidor que compruebe la credencial. Si exige autenticación, el recorrido considera la sesión del usuario. También diferencia los estados de carga, los errores y una participación ya registrada.

El servidor comprueba las condiciones de acceso y la apertura de la encuesta. La autorización para visualizar el formulario no se utiliza como sustituto de la validación al enviar: las condiciones relevantes se comprueban nuevamente en la operación de participación, porque la configuración podría haber cambiado entre ambos momentos.

1.6.3. Representación del formulario y preparación de las respuestas

`Survey` recorre los elementos ordenados de la encuesta y los de sus bloques. Los textos aportan instrucciones y contexto; las preguntas generan controles según su tipo; y los bloques agrupan partes del cuestionario. La numeración de las preguntas continúa entre los distintos niveles, mientras que los bloques mantienen su identificación visual.

Las respuestas se conservan en el estado local y se transforman antes del envío. El DTO no utiliza un único campo de texto para todas las respuestas. Una pregunta textual produce `textValue`; una numérica o de escala utiliza `numericValue`; una booleana emplea `booleanValue`; las elecciones generan `selectedOptions`; y las matrices utilizan `matrixValues`, vinculando cada valor con su fila.

En las preguntas de selección, cada opción se identifica mediante su identificador persistente. Cuando procede, se incorpora el texto asociado a una opción abierta. En una matriz se envía

el identificador de fila junto con el valor de escala. Esta estructura permite que el servidor determine a qué configuración corresponde cada dato recibido.

La validación de entrada del cliente identifica preguntas obligatorias sin responder, valores fuera de rango o selecciones incompatibles con los límites establecidos. Si se detecta un problema, se presenta junto al contenido correspondiente y se orienta el foco hacia el error. Si falla el envío por un problema de comunicación, las respuestas locales se conservan para que no sea necesario rellenar de nuevo la encuesta.

1.6.4. Registro de participaciones y coherencia de las respuestas

`ParticipationService` recibe la participación y comprueba el estado y las condiciones de acceso de la encuesta. Carga las preguntas de su estructura, incluidas las de los bloques, y contrasta con ellas las respuestas recibidas. Rechaza identificadores duplicados, preguntas ajenas a la encuesta y la ausencia de respuestas obligatorias.

La validación continúa según el tipo. En una elección, las opciones deben pertenecer a la pregunta; en una matriz, las filas deben corresponder a su configuración; y en los valores numéricos se respetan los límites y las condiciones definidas. Estas comprobaciones son necesarias aunque el formulario ya valide la entrada, pues la API puede recibir peticiones que no procedan del recorrido previsto por la interfaz.

La participación y sus respuestas se guardan de forma transaccional. El modelo separa la participación del conjunto de respuestas y utiliza relaciones específicas para opciones seleccionadas y valores de matriz. Esta organización conserva el significado de los datos y permite consultarlos posteriormente tanto de manera individual como agregada.

Para las personas autenticadas, la participación puede vincularse con su usuario. En el recorrido anónimo, `AnonymousRespondentService` utiliza una cookie HttpOnly con un token aleatorio y deriva un hash específico para la encuesta. Las restricciones únicas de la base de datos impiden repetir la combinación de encuesta y usuario, o de encuesta y hash anónimo, y respaldan la comprobación del servicio frente a envíos concurrentes.

Este mecanismo anónimo identifica el navegador que conserva el token; no constituye una identificación inequívoca de una persona entre dispositivos. Cuando se detecta una repetición, el servidor devuelve el conflicto `PARTICIPATION_ALREADY_SUBMITTED`, que la página presenta como una situación específica del recorrido de respuesta.

1.6.5. Importación de documentos estructurados

La importación conserva el objetivo inicial de aprovechar contenido preparado en un procesador de textos, concretado en la versión implementada mediante una plantilla ODT. `ImportSurveyPage` organiza el recorrido en la descarga de la plantilla, la selección o arrastre del documento y la revisión del resultado interpretado antes de continuar con la creación.

El documento debe seguir la estructura prevista por la plantilla. No se realiza una conversión general de cualquier archivo Word ni una interpretación libre de documentos arbitrarios. Esta delimitación permite reconocer preguntas, bloques y opciones de forma coherente con el modelo de FormQuid y comunicar errores cuando el contenido no se ajusta al formato esperado.

`SurveyOdtImportService` comprueba el archivo y extrae el contenido XML del contenedor ODT. La carga admite documentos de hasta cinco megabytes y la extracción aplica también un límite al contenido que procesa. El servicio recorre las estructuras reconocidas del documento y construye un `SurveyImportPreviewDto` con los metadatos y los elementos detectados.

La operación de previsualización de importación no crea una encuesta. Devuelve el resultado

de la interpretación para que el usuario lo revise. Al continuar, la página transmite ese contenido al recorrido de `CreateSurveyPage`, que permite completar la información general y confirmar la creación.

La creación importada coordina `SurveyService` y `ContentService` en una operación transaccional: primero se crea el borrador y después se incorpora su estructura. Si esta no puede guardarse, la operación no deja una encuesta parcialmente importada. A partir de ese momento, el resultado se modifica con el mismo constructor que una encuesta creada manualmente, sin mantener un editor paralelo para documentos importados.

1.6.6. Galería y almacenamiento de portadas

`SurveyCoverInput` reúne la selección de una portada desde el dispositivo, una dirección y la galería incorporada. La galería se construye a partir de los recursos gráficos de la aplicación y permite elegir una imagen sin preparar previamente un archivo externo.

Cuando se selecciona una imagen de galería, el cliente obtiene el recurso y lo convierte en un archivo que sigue el mismo recorrido de carga que una selección local. Esto reutiliza la validación y la persistencia de portadas. La galería no introduce una entidad de negocio adicional ni un procedimiento distinto para asignar la imagen a la encuesta.

El componente ofrece una vista previa y gestiona las URL temporales utilizadas para mostrar archivos locales. También informa a la página de que hay una carga en curso para evitar confirmar mientras la imagen todavía se prepara. Se admiten imágenes PNG y JPEG dentro del límite de tres megabytes previsto por el componente y el almacenamiento correspondiente.

`SurveyCoverStorage` gestiona los ficheros en un directorio externo al código y la encuesta conserva la clave que los identifica. Esta separación permite mantener los recursos mediante almacenamiento persistente al ejecutar la aplicación en contenedores. La limpieza de archivos se coordina con el resultado de las operaciones de base de datos, evitando eliminar una portada que todavía deba conservarse por un fallo en la modificación.

1.6.7. Consulta de resultados y participaciones individuales

El módulo de resultados ofrece dos niveles de lectura. `ResultsPage` permite localizar encuestas con resultados. `SurveyResultsPage` presenta el resumen agregado y el análisis por pregunta, mientras que `SurveyParticipationsPage` permite consultar participaciones concretas y desplegar sus respuestas.

El servidor comprueba el permiso de consulta y la existencia de resultados. La presentación administrativa o colaborativa de una encuesta no elimina esta comprobación sobre el recurso. Los listados de participaciones se paginan y el detalle se obtiene cuando se selecciona una participación, evitando cargar todas las respuestas individuales en la pantalla inicial.

En la vista de detalle, los componentes interpretan la respuesta según su tipo: texto, valor escalar, opciones elegidas o filas de matriz. Esta consulta reutiliza la información estructural de las preguntas para presentar respuestas comprensibles. La vista agregada, en cambio, trabaja con los indicadores calculados por el servidor y los muestra mediante componentes como `PercentageBars`, `NumericSummary`, `EvaluationBadge` y `WeightedAnalysis`.

1.6.8. Cálculo de métricas por pregunta

`StatisticsService` coordina el análisis de la encuesta y utiliza `StatisticsQueryRepository` para obtener agregados mediante consultas SQL con `JdbcClient`. Este diseño evita recuperar todas las entidades de respuesta para calcular cada indicador en Java. La carga de la estructura

identifica primero las preguntas de la encuesta y de sus bloques, y los agregados se obtienen de acuerdo con el tipo de dato.

El resumen incluye el número de participaciones y las fechas de la primera y la última. Para cada pregunta se calcula cuántas participaciones contienen respuesta y cuántas la omiten. La tasa de respuesta relaciona las respuestas presentes con el total de participaciones de la encuesta, por lo que una pregunta opcional puede tener una base de respuesta inferior al total.

En preguntas numéricas y de escala se obtienen media, mínimo, máximo, mediana y desviación típica poblacional. PostgreSQL aporta operaciones como `percentile_cont` para la mediana y `stddev_pop` para la dispersión. Las escalas también pueden representarse mediante su distribución de valores, incluyendo valores configurados que no hayan recibido respuestas cuando el tamaño de la escala permite esa representación.

Las preguntas de elección muestran frecuencias y porcentajes por opción. En una selección múltiple, una misma participación puede contabilizarse en varias opciones; por ello, la suma de los porcentajes puede superar el cien por cien. Las booleanas distinguen sus dos alternativas y las matrices desglosan las métricas por fila. El texto libre conserva su utilidad en la consulta de respuestas, sin forzar una agregación numérica que no corresponde a su naturaleza.

1.6.9. Configuración evaluativa y resultado ponderado

Además de describir respuestas, una encuesta puede asignar una interpretación evaluativa a determinadas preguntas. El constructor permite indicar dirección neutral, positiva o negativa y un peso cuando la pregunta participa en la evaluación. `QuestionService` comprueba la coherencia de estos datos: las preguntas neutrales no tienen peso, las positivas utilizan pesos de uno a cinco y las negativas de menos cinco a menos uno. Las preguntas de texto permanecen neutrales.

La normalización depende del tipo. En valores numéricos y escalas se utilizan los límites configurados; en opciones y respuestas booleanas se emplean factores de evaluación; y en matrices se consideran los valores de sus filas. En las selecciones múltiples, la agregación de factores se realiza teniendo en cuenta las opciones de cada respuesta, de modo que el significado del resultado no se reduce al número total de selecciones.

Una vez obtenido un valor normalizado n_i entre 0 y 100, el servicio lo centra y aplica la dirección de la pregunta:

$$d_i = \begin{cases} 2n_i - 100, & \text{si la dirección es positiva,} \\ -(2n_i - 100), & \text{si la dirección es negativa.} \end{cases}$$

Las preguntas neutrales no generan esta puntuación. El resultado global utiliza el valor absoluto del peso, ya que la dirección se ha aplicado en el paso anterior:

$$G = \frac{\sum_{i \in E} |w_i| d_i}{\sum_{i \in E} |w_i|},$$

donde E contiene las preguntas con puntuación y peso disponibles. Cuando no existe ninguna contribución válida, el servicio devuelve ausencia de resultado, en lugar de un cero que podría interpretarse como una evaluación neutral calculada.

Como ejemplo ilustrativo, dos preguntas con normalización 75 producen puntuaciones 50 y -50 si sus direcciones son positiva y negativa, respectivamente. Con pesos de magnitud dos y uno, el resultado global sería aproximadamente 16,67. El ejemplo muestra por qué el signo de la dirección y la magnitud del peso tienen responsabilidades distintas dentro del cálculo.

Esta información se presenta separada de las métricas descriptivas. Una frecuencia, una media y una puntuación ponderada responden a preguntas diferentes; mantener esa distinción permite

interpretar el análisis sin confundir un recuento de respuestas con la valoración configurada por el autor.

1.6.10. Notificaciones y comunicación por correo

El módulo de notificaciones conecta acciones de registro, propuestas de catálogo, colaboración y propiedad. `NotificationService` crea los avisos asociados a sus receptores, y la cabecera muestra el número pendiente de lectura. `NotificationsModal` permite consultar los avisos, filtrar por lectura, marcar uno o todos como leídos y eliminarlos.

La actualización del contador se apoya en consultas cuando la ventana recupera el foco y en eventos del cliente que indican cambios en las notificaciones. El mecanismo no utiliza un canal WebSocket. Al seleccionar un aviso, el cliente obtiene el destino correspondiente y navega hacia el recurso, considerando que este puede haber cambiado o dejado de estar disponible desde que se generó el mensaje.

El correo complementa los avisos internos en los procesos que lo requieren. El envío se coordina mediante `NotificationEmailDispatcher`. Este componente atiende eventos después de confirmar la transacción y delega el envío en un ejecutor, utilizando `JavaMailSender` y la configuración SMTP. El servicio contempla reintentos ante fallos de envío y mantiene la configuración del correo fuera de la lógica de las páginas.

La confirmación de la operación principal no depende de que el servidor SMTP responda correctamente. Por ejemplo, aprobar una solicitud no debe deshacerse porque falle después el correo informativo. La separación entre la transacción de negocio y la entrega del mensaje permite conservar esta coherencia y tratar el envío como una operación posterior.

1.6.11. Componentes comunes e interacción

La finalización de los recorridos también requiere componentes que mantengan un comportamiento consistente. Los listados comparten búsqueda, filtros, tablas y paginación; las acciones sensibles utilizan diálogos de confirmación; y las cargas y errores se presentan mediante componentes comunes. Esto reduce las diferencias de interacción entre catálogo, usuarios, encuestas y participaciones.

El gancho `useModalFocus` gestiona el foco inicial, su recorrido dentro del diálogo y su devolución al elemento que lo abrió. También atiende el cierre mediante Escape y coordina los diálogos apilados. Junto con las alternativas de ordenación del constructor y los mensajes asociados a los campos, estas medidas facilitan el uso mediante teclado en recorridos que contienen formularios y ventanas superpuestas.

1.6.12. Resultado de la iteración

El cierre de este bloque conectó la construcción de encuestas con su uso efectivo. Una encuesta puede prepararse manualmente, a partir del catálogo o mediante importación, personalizarse, abrirse y recoger respuestas conforme a sus condiciones de acceso. Las participaciones alimentan después la consulta individual, las métricas descriptivas y, cuando se configura, el análisis ponderado.

1.7. Revisión global del proceso de desarrollo

El trabajo desarrollado durante estos meses fue ampliando la plataforma alrededor de un mismo modelo de contenido. Las preguntas sostienen los bloques; los bloques y las preguntas

forman plantillas; las plantillas y los elementos del catálogo sirven como origen para encuestas independientes; y las respuestas se vinculan con la estructura concreta que se ha publicado. Esta continuidad explica la importancia de las copias y de la restricción de cambios estructurales cuando ya existen participaciones.

La revisión del Sprint 1 tuvo un efecto que se extendió a toda la implementación. Separar ciclo de vida, acceso y visibilidad permitió expresar condiciones de publicación sin multiplicar estados ambiguos. Unificar las propuestas de catálogo facilitó incorporar distintos tipos de elementos mediante un mismo procedimiento de revisión. Mantener las etiquetas dentro de la encuesta evitó desarrollar una administración independiente que no aportaba una necesidad funcional equivalente.

En la interfaz, la composición de constructores redujo la duplicación de las operaciones de edición. `SurveyBuilder` configura el comportamiento de `TemplateBuilder`, y este integra las capacidades de bloques y preguntas. En el servidor, los servicios conservan las validaciones compartidas y coordinan las operaciones persistentes. La importación y la aprobación de propuestas aprovechan esos mismos servicios, de forma que el origen del contenido no cambia las reglas que debe cumplir.

Otra parte relevante del desarrollo fue resolver operaciones que atraviesan varios módulos. Crear una encuesta con portada exige coordinar peticiones y reintentos; transferir su propiedad modifica relaciones de usuario y colaboración; registrar una participación condiciona las posibilidades futuras de edición; y aprobar una solicitud produce cambios persistentes y comunicaciones posteriores. Documentar estas relaciones permite comprender el trabajo de integración que no resulta visible al enumerar únicamente las pantallas.

El resultado es una aplicación en la que autoría, reutilización, colaboración y análisis comparten reglas y estructuras. La organización por iteraciones permitió desarrollar primero los mecanismos comunes y completar después los recorridos que dependen de ellos, manteniendo una relación identificable entre las historias de usuario y las funcionalidades implementadas.

1.8. Despliegue de la aplicación

El despliegue de FormQuid exigió coordinar la interfaz, la API, la base de datos y los archivos que suben los usuarios. Estos elementos tienen necesidades diferentes: la interfaz puede distribuirse como archivos estáticos, el servidor debe ejecutar las reglas de negocio y las migraciones, y los datos deben conservarse entre nuevas versiones. Por ello, la preparación del entorno local y la publicación de la aplicación se documentan como configuraciones relacionadas, pero con alcances distintos.

1.8.1. Contenedores para el entorno de desarrollo

El archivo `docker-compose.yml` permite iniciar conjuntamente el backend, la interfaz y MailHog. El backend se construye con Java 21 y ejecuta el perfil `dev`; el frontend utiliza Node.js 22 y sirve la aplicación mediante el servidor de desarrollo de Vite. MailHog proporciona un destino para comprobar el correo sin enviarlo a destinatarios reales. PostgreSQL queda fuera de este conjunto y debe estar disponible para el backend mediante una conexión TLS.

La configuración se proporciona a través de un archivo `.env` excluido del repositorio. En él se indican la conexión a la base de datos, la clave de firma de las sesiones y, cuando corresponde, los datos de la primera cuenta administradora. Los archivos subidos se guardan fuera de la imagen del servidor: los volúmenes `survey-covers` y `profile-photos` conservan, respectivamente, las portadas de encuestas y las fotografías de perfil. Esta separación permite reconstruir los contenedores sin incorporar los archivos de los usuarios a la imagen.

Existe un archivo adicional, `docker-compose.prod.yml`, que cambia el perfil del backend a `prod` y exige el origen de la interfaz y los datos del superadministrador inicial. Antes de utilizarlo, `check-prod-env.py` comprueba que las variables requeridas no estén vacías y que `FRONTEND_ORIGIN` sea un origen HTTPS concreto, sin mostrar los secretos. La combinación de ambos archivos sirve para ejecutar el backend con su configuración de producción, pero conserva el servicio de Vite del Compose principal. Por tanto, no constituye por sí sola la configuración completa de publicación de la interfaz.

1.8.2. Publicación de la interfaz y de la API

La configuración de publicación separa el frontend en Vercel y el backend en Render. Al compilar la interfaz, el cliente utiliza por defecto la ruta relativa `/api/v1` para sus peticiones en producción. El archivo `vercel.json` reenvía las rutas `/api` a la API alojada en Render y dirige las demás rutas de la aplicación a `index.html`. Esta segunda regla permite abrir directamente páginas gestionadas por React Router. La regla de la API evita almacenar en caché las respuestas reenviadas, un detalle relevante para peticiones que dependen de la sesión y del estado actual de los datos.

El backend mantiene sus propias variables de entorno en el servicio donde se ejecuta. Necesita los datos de conexión a PostgreSQL, `JWT_SECRET` y `FRONTEND_ORIGIN`; el perfil `prod` requiere además los cuatro datos de la cuenta inicial de superadministrador. La URL JDBC exige TLS mediante `sslmode=require`. Al arrancar, Flyway aplica las migraciones del esquema y JPA no intenta generarlo. El inicializador crea el primer superadministrador solo cuando la base todavía no contiene usuarios, de modo que cambiar posteriormente esas variables no modifica una cuenta existente.

La sesión se transporta en una cookie HttpOnly que el perfil de producción marca como segura. El origen de la interfaz se utiliza en la configuración de CORS y las operaciones que modifican datos siguen utilizando la protección CSRF descrita en la implementación. La URL pública del frontend también interviene en los enlaces de recuperación de contraseña. El correo saliente se configura en el backend mediante variables SMTP; la variante local de Compose para Brevo no traslada automáticamente esa configuración al servicio de Render.

1.8.3. Persistencia y mantenimiento

La base PostgreSQL y los archivos subidos tienen que mantenerse al margen de las reconstrucciones del backend. En Compose, los dos directorios de archivos están respaldados por volúmenes con nombre; en otro alojamiento deben utilizarse rutas con almacenamiento persistente configuradas mediante `SURVEY_COVER_STORAGE_PATH` y `PROFILE_PHOTO_STORAGE_PATH`. Una copia de seguridad útil debe abarcar tanto la base de datos como las portadas y fotografías. Antes de actualizar una base existente también es necesario revisar el historial de Flyway y disponer de una copia, especialmente si ya se aplicó una versión anterior de la migración V9, cuyo checksum puede diferir.

El repositorio incluye además un flujo de GitHub Actions para registrar los despliegues asociados a un commit. Cuando cambia el frontend o el backend, el flujo consulta el resultado comunicado por Vercel o Render antes de registrar el estado correspondiente en GitHub; también admite una ejecución manual para un commit concreto. Este registro aporta trazabilidad entre la versión del código y el resultado del proveedor, sin sustituir la comprobación funcional de la aplicación publicada ni la vigilancia de sus datos y archivos persistentes.